The Memory Conservatory

README.MD · SEPTEMBER 9, 2026

Static marketing site for a Chennai-based wedding photographer. Plain HTML, CSS and JavaScript with Tailwind for utilities. No framework, no runtime, no hydration. Deployed on Netlify.

Running it

```
npm install
npm run dev
```

`npm run dev` builds once, then watches CSS and HTML and serves the site at http://localhost:3000.

Script	What it does
`npm run build`	Full production build (CSS + HTML)
`npm run dev`	Build, watch and serve on port 3000
`npm run build:css`	Tailwind only
`npm run build:html`	Partial expansion only

Note that a build step is required even though the site is plain HTML: Tailwind generates the stylesheet, so opening a file directly from disk will show unstyled markup. Use `npm run dev`.

Layout

```
partials/          header, footer, <head> — shared by every page
src/pages/         one authored file per page
src/input.css      THE design system: every colour, font and size, defined once
public/            the deployed site — Netlify's publish directory
  assets/js/preferences.js   <- site configuration, see below
build.js           expands @include comments into plain HTML
```

Authored pages live in `src/pages/`. `public/*.html` is generated — edit the source, not the output, or your change will be overwritten on the next build.

How pages are assembled

`build.js` replaces `<!-- @include name -->` with `partials/name.html`:

```
<!-- @include head title="Gallery" description="..." -->
<!-- @include header active="gallery" -->
<!-- @include footer -->
```

Attributes become `{{placeholders}}` inside the partial. The result is ordinary static HTML — no templating survives into the browser. This matters for Netlify Forms, which only detects forms physically present in the built HTML.

Change the nav once in `partials/header.html` and every page picks it up.

Pages

Source	URL	State
`src/pages/index.html`	`/`	Built. Copy still partly Lorem Ipsum.
`src/pages/about.html`	`/about`	Built. **All copy is the client's own, verbatim.**
`src/pages/contact.html`	`/contact`	Built. Netlify Forms, all 11 client questions.
`src/pages/thank-you.html`	`/thank-you`	Built. Where the form posts on success.
`src/pages/faq.html`	`/faq`	Built. **All 12 Q&As are the client's own, verbatim.**
—	`/gallery`	**Not built — 404.**

The nav's **The Process** and **Testimonials** are not pages: they are fragment links to `#process` and `#testimonials` on the home page. They are written as `/#process`, not `#process`, so they still work from `/about`, `/faq` and the enquiry form — a bare fragment there would look for a section that page has not got. `section[id] { scroll-margin-top: 4.75rem }` keeps the target clear of the fixed header.

Interior pages open with `.page-hero` instead of the landing `.hero`: a shorter band with no carousel, no `preferences.js` bridge and no Ken Burns. Using it also switches the site header to a solid Deep Blue bar (`body:has(.page-hero) .site-header`), because Soft Cream nav text measured **2.0:1** against the bright parts of the photographs behind a transparent one.

Two cross-links live in partials so their wording exists in exactly one place:

- `partials/about-link.html` — "*If our work resonates with you…*", for every gallery page (currently on `/` and `/thank-you`).
- `partials/enquiry-link.html` — "*Start a conversation about your wedding…*", at the bottom of `/about` and `/faq`.
- `partials/faq-link.html` — "*We've put together a few thoughtful answers…*", at the bottom of `/about`. Uses `.cross-link--block`, a panel variant: the inline cross-link underlines its own baseline, which falls apart once the wording runs to two sentences.

- `partials/testimonials.html` — the nine client reviews, for the home page and the end of every gallery page.

The testimonials are a CSS scroll-snap track, not a JS slider: swipe, trackpad flick and keyboard scrolling all work before `main.js` loads, and the arrows only call `scrollBy()`. Three across on desktop, two on a tablet, one on a phone — so nine reviews page as 3, 5 or 9 screens, measured from the real card width rather than assumed. `main.js` also sizes the track to the tallest card on screen; one review is twice the length of the others and would otherwise leave the shorter pages about 45% empty.

The FAQ accordion is native `<details>` / `<summary>` — no JavaScript, no ARIA to get wrong, and every answer stays readable if the script never runs. The CSS only replaces the default disclosure triangle.

Configuration — `public/assets/js/preferences.js`

Behaviour is configured there, not in markup or CSS. It is organised per page so each new page adds its own section.

Currently:

- `site.whatsapp` — the WhatsApp enquiry button. `enabled: false` removes it from every page at once.
- `home.isBlackAndWhite` — hero photography in black & white (`true`) or colour (`false`). A display filter; source files are untouched.
- `home.imageSrcListForCarousel` — **always an array.** More than one image renders a carousel; exactly one renders a static background with no carousel behaviour and no timer. Length alone decides.
- `home.carouselIntervalMicroseconds` — hold time per image in **microseconds** (1 second = 1000000). Ignored entirely when there is only one image.
- `home.tintOpacity` — flat black tint on the hero photograph, beneath the gradient and grain. Applied in colour mode as well as black & white. Default `0.18`.
- `home.veilOpacity` — flat black veil between the photograph and the content, above the gradient and grain. Lifts the headline and buttons off a busy or bright image. Default `0.14`.
- `home.grainOpacity` (0-1), `home.grainContrast` (>=1), `home.grainBlendMode` — film grain. **Opacity is clamped to 1 by the CSS spec**, so if the grain looks weak at `1` the fix is `grainContrast`, or switching `grainBlendMode` to `screen`: `overlay` modulates midtones and collapses toward black as the backdrop darkens, so a heavily tinted hero suppresses it no matter how opaque the layer is.
- `home.kenBurns` — slow zoom on the hero image.

The file is commented in full. It is served directly to the browser, so editing it needs no rebuild — just reload.

Design system

All tokens live in `src/input.css`. **Never hardcode a hex value in a page.**

Colour — the official TMC palette

> ***One deliberate exception.*** *`--color-gold: #c8a24a` is not in the brand palette. It is the pre-rebrand gold, restored at the client's explicit request, and is scoped to exactly one thing: the italic `<em>` inside a headline (`.section__title em`, `.hero__title em`, `.page-hero__title em`). Soft Cream measures 1.2:1 on White and vanishes there, so without this the emphasis fell back to plain Deep Blue and read as no accent at all. The trade-off, measured: gold is 3.7:1 on Deep Blue, 2.4:1 on White and 2.0:1 on Soft Cream — below the 3:1 AA threshold for large text on the two light grounds. `#a9822f` would clear it.* ***Nothing else may use this token.***

Four colours, exactly as specified in the client's brand guidelines. The guidelines are explicit: "*Do not use colors that are not included in the brand's official palette for brand representation.*"

	Hex	Role per the guidelines
Soft Cream	`#FFE9B0`	Warmth, softness, positivity
Deep Blue	`#144C78`	**Primary.** Trust, confidence, professionalism. Used prominently.
White	`#FFFFFF`	Clarity, openness, contrast. Background.
Grey	`#9B9B9B`	Secondary information, subtle backgrounds, supporting elements

Nothing else is introduced. Where a softer or deeper value is needed the design uses one of these four at reduced opacity — the guidelines set that precedent with their own 20/40/60/80% cream tint ramp.

Two traps, both measured and handled:

- **Soft Cream is a near-highlight, not a mid-tone.** It cannot be used for text on White or on the Soft Cream band — it disappears. Accents are Deep Blue on light grounds and flip to Soft Cream on Deep Blue grounds and over photography.
- **Grey is 2.8:1 on White**, below the 4.5:1 body-copy threshold. It is not used for running text; secondary text is Deep Blue held back with opacity instead.

Every text style on the page is contrast-audited: **18/18 pass WCAG AA.**

Typography

The guidelines specify **Dream Avenue** (headers) and **ITC Avant Garde** (body, Regular + Bold).

> ***Neither font file was supplied and both are licensed commercial faces.*** *The site currently ships documented substitutes — Playfair Display standing in for Dream Avenue, Poppins for ITC Avant Garde — chosen to sit as close as possible to the specimens.*
>
> *To swap in the real fonts: drop the licensed `.woff2` files into `public/assets/fonts/`, change the two `src: url()` lines in the `@font-face` block at the top of `src/input.css`, and update `--font-display` / `--font-sans`. Nothing else needs to change — no component references a font family directly; they all go through the two tokens.*

Logo

The client's supplied artwork lives in `public/assets/brand/`, trimmed of its transparent margins and exported at web sizes. **Do not recolour, redraw or respace it** — the guidelines forbid altering the logo. Only its size and clear space are controlled in CSS.

File	Use
`wordmark-cream.png`	On Deep Blue and over photography (header, footer)
`wordmark-blue.png`	On White and Soft Cream grounds
`wordmark-black.png`	Monochrome / high-contrast applications
`monogram-blue/cream.png`	The vertical TMC mark
`tagline-cream/black.png`	"Preserving love, frame by frame." lockup
`favicon-*.png`, `apple-touch-icon.png`	Generated from the monogram on Deep Blue

Responsive

Built and tested from **320px to 2560px**. Content containers cap (1440px standard, ~68 characters for prose) while full-bleed imagery spans the viewport; grids gain columns rather than inflating. Type scales continuously with `clamp()`. `100svh` is used instead of `100vh` so mobile browser chrome cannot clip the hero. Nothing may cause a horizontal scrollbar at any width.

Deployment

Netlify builds with `npm run build` and publishes `public/` — see `netlify.toml`.

The enquiry form

`/contact` posts to Netlify Forms. Three things break it silently, so all three are worth knowing:

1. **Netlify only registers forms it finds in the deployed HTML.** This is why `build.js` inlines partials rather than injecting them with JavaScript.
2. **The hidden `form-name` input must match the form's `name`** (`enquiry`), or submissions are accepted and filed nowhere.
3. **It cannot be tested locally.** Locally the form validates and then posts to a URL that does not handle it.

After the first deploy: submit once, confirm the entry under **Site configuration → Forms**, confirm the couple lands on `/thank-you`, and turn on an email notification so enquiries are not sitting unread in the dashboard.

`netlify-honeypot="bot-field"` names the hidden field bots fill in; it is inside an `.sr-only` wrapper, so people never see it.

Placeholder content is marked `<!-- TODO:CLIENT -->`. See `PRELAUNCH-CHECKLIST.md` for everything that must be replaced before launch.

TheMemoryConservatory